Coding and Testing

S. S Satapathy

Coding

- **Convert** the **design** **of a system** into **code** **written in some high level language**
- **Unit test** the **code**

Coding Standards

- **Good software development require** programmers to adhere to some **well-defined** and **standard style** of coding called **coding standards**.
- Most **software development organizations formulate their own coding standards**.
- **Benefit of** following a **coding standard** are:
  - **Uniform appearance** to the codes written by different programmers
  - **Better code understanding**
  - Encourages **good programming practices**
- A **coding standard lists several rules to be followed** during coding, such as the way variables are to be named, the way the code is to be laid out, error return conventions, etc

Representative Coding standards

- **Rules for limiting** the **use of global data**:
  - These rules list what types of data can be declared global and what cannot.
- **Contents of the headers** preceding codes for different **modules**:
  - Some standard header data:
    - **Name** of the module
    - **Date** on which the module was **created**
    - **Author's name**
    - **Modification history**
    - **Synopsis** of the module
    - Different **functions supported**, along with their **input/output parameters**
    - **Global variables accessed/modified** by the module
- **Naming conventions** for **global variables**, **local variables**, and **constant identifiers**:
  - Examples: global variable names always start with a capital letter, local variable names are made of small letters, and constant names are always capital letters.

Representative Coding standards

- **Error return conventions** and **exception handling** mechanisms:
  - The way error conditions are reported by different functions in a program are handled should be standard within an organization. For example, different functions while encountering an error condition should either return a 0 or 1 consistently.

Representative Coding Guidelines

- **Do not use** a **coding style** that is **too clever** or **too difficult to understand**:
  - **Code should be easy to understand**.
  - **Cryptic and incomprehensible** code **hamper understanding** and makes maintenance difficult
- **Avoid obscure side effects**:
  - The **side effects of a function call** include **modification of parameters passed by reference**, **modification of global variables**, and **I/O operations**.
  - An **obscure side effect is one that is not obvious from a casual examination of the code**.
  - **Obscure side effects make it difficult to understand** a piece of code.

Representative Coding Guidelines

- **side effects**:

```
//Side Effect
#include<stdio.h>
int main()
{
   int d[10]={1,2,3,4,5,6,7,8,9,10},n=100;
   d[10]=0;
   printf("Value of n=%d",n);
   return(0);
}
```

Representative Coding Guidelines

- **side effects**:

```
//Side effect
#include<stdio.h>
void function(int *,int *);
int main()
{
   int a=10,b=20;
   function(&a,&b);
   printf("Sum of a and b=%d",a+b);
   return(0);
}
void function(int *a,int *b){
  XXXXXXXX
   *a=100;
   *b=200;
}
```

Representative Coding Guidelines

- **Do not use an identifier for multiple purposes**:
  - Programmers often use the same identifier to denote several temporary entities.
- The **code should be well-documented**:
  - As a rule of thumb, there must be at least one comment line on the average for every three-source line.
- The **length of any function should not exceed 10 source lines**:
  - A function that is very lengthy is usually very difficult to understand
    - it may carries out many different functions.
    - likely to have disproportionately larger number of bugs.
- **Do not use GOTO statements**:
  - **Use of GOTO** statements **makes a program unstructured** and makes it very difficult to understand

Coding Standards followed in Industry

- **BARR-C** (By **Michael Barr**) coding standard used for **embedded software development and quality assurance processes**.
- **The MISRA C** (2012): – Guidelines for the **Use of the C Language in Critical Systems**
  - **MISRA** stands for :**Motor Industry Software Reliability Association**
  - This standard is **used for designing products that could kill or injure people**.
  - It **defines a subset of the C programming language that is safer**
  - It is **more restrictive than** the rules in this **BARR-C** coding standard. **MISRA-C++** is also available for coding in C++.

Some of the BARR-C Coding Standards

- **Braces {}**
  - **Braces shall always surround the blocks of code** (a.k.a., compound statements), following if, else, switch, while, do, and for statements; single statements and empty statements following these keywords shall also always be surrounded by braces.
  - Each **left brace ({) shall appear** by itself on the **line below the start of the block it opens**. The corresponding **right brace (}) shall appear** by **itself in the same position** the appropriate number of lines later in the file.
  - Example:

```
{
  if (x>0)
  y=0;
  else
  y=1;
  z=1;
}
// This is legal...
```

```
{
  if (x>0)
  {
    y=0;
  }
  else
  {
    y=1;
  }
  z=1;
}          // This is Better
```

Some of the BARR-C Coding Standards

- **Parentheses ()**
  - a. Do not rely **on C's operator** precedence rules, as **they may not be obvious** to those who maintain the code. **To aid clarity**, **use parentheses** (and/or break long statements into multiple lines of code) to ensure proper execution order within a sequence of operations.
  - b. **Unless it is a single identifier or constant**, **each operand** of the logical AND (&&) and logical OR (||) operators shall be **surrounded by parentheses**.
  - Example:

```
if ((depth_in_cm > 0) && (depth_in_cm < MAX_DEPTH))
{
        depth_in_ft = convert_depth_to_ft(depth_in_cm);
}
```

Some of the BARR-C Coding Standards

- **Comment Rules**
  - a. Single-line **comments** in the C++ style (i.e., preceded by //) **are a useful and acceptable** alternative to traditional C style comments (i.e., /* … */).
  - b. **Comments shall never contain the preprocessor tokens** /*, //, or \.
  - c. **Code shall never be commented out**, even temporarily.
    - **To temporarily disable** a block of **code**, **use the preprocessor's conditional compilation feature**.

```
#include <stdio.h>
#define UNIX 1
int main()
{
        #ifdef UNIX
        printf("UNIX specific function calls go here.\n");
        #endif
        …..
        return 0;
}
```

Some of the BARR-C Coding Standards

- **White Space Rules**
  - **Keywords** if, while, for, switch, and return **shall be followed by one space**.
  - **Assignment operators** =, +=, -=, *=, /=, %=, &=, |=, ^=, ~=, and != shall always be **preceded and followed by one space**.
  - **Binary operators** +, -, *, /, %, <, <=, >, >=, ==,!=, <<, >>, &, |, ^, &&, and || shall **always be preceded and followed by one space**.
  - **Unary operators** +, -, ++, --, ! , and ~, shall be written **without a space on the operand side**.
  - The **pointer operators** *** and &** shall be written **with white space on each side within declarations BUT otherwise without a space on the operand side**.
  - The ? and : characters (**ternary operator**) shall each be **preceded and followed by one space**.
  - The **structure pointer** **(->)** and **member operators (.)** shall be **without surrounding spaces**.
  - The left and right brackets of the array **subscript operator** **([and])** **shall be without surrounding spaces**, except as required by another white space rule.
  - **Expressions within parentheses** shall always have **no spaces adjacent to the left and right parenthesis** characters.
  - The **left and right parentheses of the function call operator** shall always be **without surrounding spaces**, except that the function declaration shall feature one space between the function name and the left parenthesis to allow that one particular mention of the function name to be easily located.
  - Except when at the end of a line, each **comma separating function parameters** shall always **be followed by one space**.
  - **Semicolon** **separating** the **elements** of a **for statement** shall always **be followed by one space**.

```
int a, b, c;
int* p1;
Int * p2;
p1 = &a;
c = *p1;
p2 = &b;
```

Some of the BARR-C Coding Standards

- **Alignment Rules**
  - The **names of variables within a series of declarations** shall have their **first characters aligned**.
  - The names of **struct and union members** shall have their **first characters aligned**.
  - The assignment operators within a block of adjacent assignment statements shall be aligned.

```
int x;
float y;
char z;
…
typedef struct
{
        uint8_t buffer[BUFFER_BYTES];
        uint8_t checksum;
} string_t;
```

Some of the BARR-C Coding Standards

- **Blank Lines**
  - **No line of code shall contain more than one statement**.
  - There shall be a **blank line before** and **after** each **natural block of code**.
  - Examples of **natural blocks** of code are **loops**, **if…else** and **switch** statements, and **consecutive declarations**.
  - Each **source file shall terminate with a comment marking the end of file followed by a blank line**.

Some of the BARR-C Coding Standards

- **Indentation**
  - Each **indentation level should align at a multiple of 4 characters** from the start of the line.
  - **Within a switch statement, the case labels shall be aligned**; the contents of each case block shall be indented once from there.
  - **Whenever a line of code is too long to fit within the maximum line width, indent the second and any subsequent lines in the most readable manner possible**.

```
sys_error_handler(int err)
{
        switch (err)
        {
                case ERR_THE_FIRST:
                ...
                break;
                default:
                ...
                break;
        }
}
```

Some of the BARR-C Coding Standards

- **Naming Conventions**
  - All **module names** shall consist of **lowercase letters**, **numbers**, and **underscores**. No spaces shall appear in names.
  - All **module names** shall be **unique in their first 8 characters** and **end with** suffices **.h** and **.c** for the **header** and **source** file names, respectively.
  - No module's header file name shall share the name of a header file from the C Standard Library or C++ Standard Library. For example, **modules shall not** be named "**stdio**" or "**math**".
  - Any **module containing a main() function** shall have the word "**main**" **as part of its source file name**.

Some of the BARR-C Coding Standards

- **Header Files**
  - There shall always be **precisely one header file for each source file** and they shall always have the **same root name**.
  - Each **header file shall contain a preprocessor guard against multiple inclusion**, as shown in the example below.

```
#ifndef ADC_H
#define ADC_H
...
#endif /* ADC_H */
```

  - The **header file shall identify only the procedures, constants, and data types** (via prototypes or macros, #define, and typedefs, respectively) about which it is strictly necessary for other modules to be informed.
    - It is a preferred practice that **no variable ever be declared (via extern) in a header file**.
    - **No storage for any variable shall be allocated in a header file**.
  - **No public header file shall contain a #include of any private header file**.

Some of the BARR-C Coding Standards

- **Source Files**
  - Each **source file shall be comprised of** some or all of the **following sections**, **in the order** listed:
    - comment block;
    - include statements;
    - data type, constant, and macro definitions;
    - static data declarations;
    - private function prototypes;
    - public function bodies;
    - then private function bodies.
  - Each **source file shall always #include the header file of the same name** (e.g., file adc.c should #include “adc.h”), to allow the compiler to confirm that each public function and its prototype match.
  - **Absolute paths shall not be used in include file names**.
  - Each **source file shall be free of unused include files**.
  - **No source file shall #include another source file**.

Some of the BARR-C Coding Standards

- **Type Casting**
  - a. Each cast shall feature an associated comment describing how the code ensures proper behavior across the range of possible values on the right side.
  - Example:

```
int abs (int arg)
{
        return ((arg < 0) ? -arg : arg);
}
...
unsigned sample = read();
result = abs((int) sample);
```

  - **Casting is dangerous**. In the example above, unsigned "sample" can hold larger positive values than a signed integer. In that case, the absolute value will be incorrect. Thus there is a possibility of overflow error.
    - E.g., in an 5bit system, integer range is 15 to -16 and unsigned int range is 0 to 31. If *sample* is assigned value 17, then the return value of *abs* will be 15.

Some of the BARR-C Coding Standards

- **Data Type Rules**
  - Fixed-Width Integers Rules:
    - Whenever **the width** (in bits/bytes) **of an integer value matters** in the program, one of the **fixed width data types shall be used** in place of char, short, int, long, or long long.

Integer Width	Signed Type	Unsigned Type
8 bits	int8_t	uint8_t
16 bits	int16_t	uint16_t
32 bits	int32_t	uint32_t
64 bits	int64_t	uint64_t

    - The **keywords short and long shall not be used**.
    - Use of the keyword **char shall be restricted to the declaration of and operations concerning strings**.

Some of the BARR-C Coding Standards

- **Data Type Rules**
  - **Signed and Unsigned Integers:**
    - **Bitwise operators** (i.e., &, |, ~, ^, <<, and >>) shall **NOT** be **used to manipulate signed integer** data.
    - **Signed integers shall not be combined with unsigned integers in comparisons or expressions**.
  - **Floating Point Rules:**
    - **Avoid** the use of **floating point constants and variables** whenever possible. **Fixed-point math may be an alternative**.
    - When floating point calculations are necessary:
      - Use the C99 type names float32_t, float64_t, and float128_t.
      - Append an 'f' to all single-precision (32bit) constants (e.g., pi = 3.141592f).
      - Ensure that the compiler supports double precision, if your math depends on it.
      - **Never test for equality or inequality of floating point values.**
      - Always **invoke the isfinite() macro to check that prior calculations have resulted in neither INFINITY nor NAN**.

Some of the BARR-C Coding Standards

- **Functions Rules:**
  - Keep the **length of each function limited to one printed page**, or a maximum of 10 lines.
  - Whenever possible, **all functions** shall be made to **start at the top of a printed page**, **except** when several **small functions** can fit onto a single page.
  - It is a preferred practice that **all functions shall have just one exit point** and it shall be **via a return at the bottom of the function**.
  - A **prototype shall be declared for each public function** in the module header file.
  - All **private functions shall be declared static**.
  - Each **parameter** shall be **explicitly declared and meaningfully named**.

Some of the BARR-C Coding Standards

- **Function like MACRO Rules:**
  - **Parameterized macros shall not be used if a function can be written to accomplish the same behavior**.

```
// Don't do this ...
#define MAX(A, B) ((A) > (B) ? (A) : (B))

// ... when you can do this instead.
inline int max(int num1, int num2)
```

  - **If parameterized macros are used for some reason**, these rules apply:
    - **Surround the entire macro body with parentheses**.
    - **Surround each use of a parameter with parentheses**.
    - **Use each parameter no more than once, to avoid unintended side effects**.
    - **Never include a transfer of control (e.g., return keyword)**.

Some of the BARR-C Coding Standards

- **Function like MACRO Rules:**
  - **Parameterized macros shall not be used if a function can be written to accomplish the same behavior**.

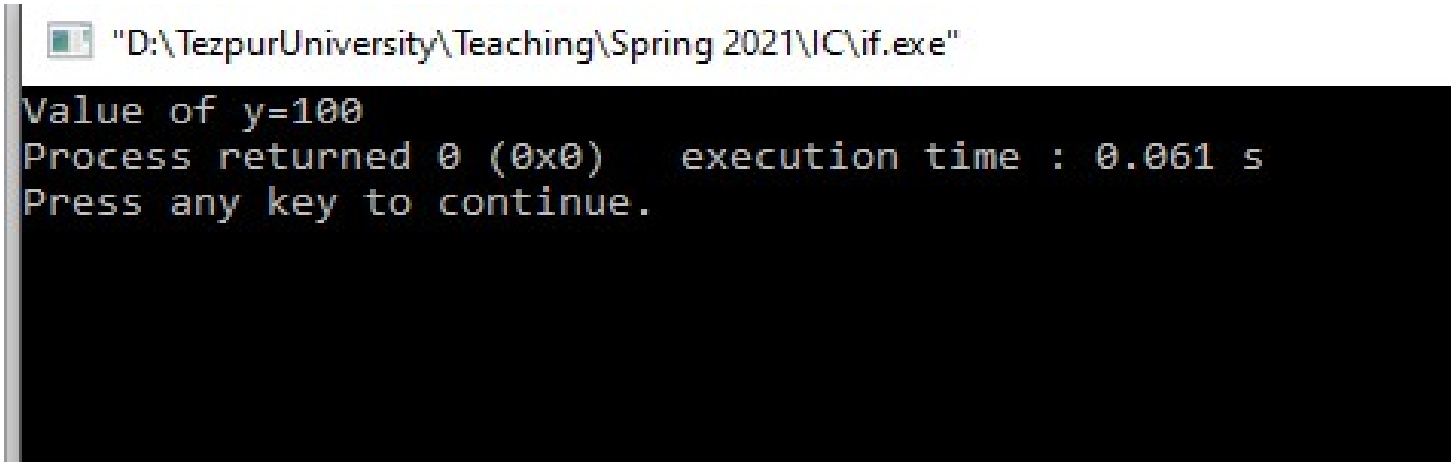
```
// Don't do this ...
#define SQR(X) ((X)*(X))

a=SQR(10+10);
```

  - **If parameterized macros are used for some reason**, these rules apply:
    - **Surround the entire macro body with parentheses**.
    - **Surround each use of a parameter with parentheses**.
    - **Use each parameter no more than once, to avoid unintended side effects**.
    - **Never include a transfer of control (e.g., return keyword)**.

Some of the BARR-C Coding Standards

```
#include<stdio.h>
#define sqr(x) x*x
int main()
{
   int y;
   y=sqr(10);
   printf("Value of y=%d",y);
   return(0);
}
```



Some of the BARR-C Coding Standards

```
#include<stdio.h>
#define sqr(x) x*x
int main()
{
   int y;
   y=sqr(5+5);
   printf("Value of y=%d",y);
   return(0);
}
```

```
"D:\TezpurUniversity\Teaching\Spring 2021\IC\if.exe"
Value of y=35
Process returned 0 (0x0)   execution time : 0.229 s
Press any key to continue.
```

Some of the BARR-C Coding Standards

- **Common Abbreviations**
  - a. **Abbreviations and acronyms** should generally be **avoided** unless their meanings are widely and consistently understood in the engineering community.
  - b. A **table of** project-specific **abbreviations** and **acronyms** shall be **maintained** in a version-controlled document.

Some of the BARR-C Coding Standards

- **Keywords to Avoid**
  - The **auto** keyword shall not be used. The auto keyword is an unnecessary historical feature of the language.
  - The **register** keyword shall not be used. The register keyword **presumes the programmer is smarter than the compiler**.
  - It is a preferred practice to **avoid all use of the goto** keyword. Their use too often results in **spaghetti code (code with noodle like control structure)**. If goto is used it shall only jump to a label declared later in the same or an enclosing block.
  - It is a preferred practice to **avoid all use of the continue** keyword.

Some of the BARR-C Coding Standards

- **Keywords to use frequently**
  - The **static** keyword shall be used to declare all functions and variables that do not need to be visible outside of the module in which they are declared.
  - The **const** keyword shall be used whenever appropriate. Examples include:
    - To declare **variables that should not be changed after initialization**
    - To define **call-by-reference function parameters that should not be modified** (e.g., char const * param)
    - To define **fields in a struct or union that should not be modified** (e.g., in a struct overlay for memory-mapped I/O peripheral registers), and
    - As a **strongly typed alternative to #define for numerical constants**.

Some of the BARR-C Coding Standards

- **Initialization Rule:**
  - All **variables shall be initialized before use**.
  - It is preferable to **define local variables as you need** them, **rather than all at the top** of a function.
  - **If project- or file-global variables are used**, their definitions shall be grouped together and **placed at the top of a source** code file.
  - Any **pointer variable lacking an initial address** shall be **initialized to NULL**.
- **Variable Declarations Rules:**
  - The **comma operator (,) shall not be used within variable declarations**.

Example:

char * x, y; // Was y intended to be a pointer also? Don't do this.

Reasoning: The cost of placing each declaration on a line of its own is low. By contrast, the risk that either the compiler or a maintainer will misunderstand your intentions is high.

Some of the BARR-C Coding Standards

- **Conditional Statements Rule:**
  - It is a preferred practice that the **shortest (measured in lines of code)** of the if and else if clauses **should be placed first**.
  - **Nested if…else** statements shall **not be deeper than two levels**. Use function calls or switch statements to reduce complexity and aid understanding.
  - **Assignments shall not be made within an if or else if test**.
  - **Any if statement with an else if** clause shall **end with an else** clause. This is the **equivalent of requiring a default case** in every switch statement

```
if (NULL == p_object) {
        result = ERR_NULL_PTR;
}
else if (p_object = malloc(sizeof(object_t))) {// No assignments!
        ...
}
else{
        // Normal processing steps, which require many lines of code.
        ...
}
```

Some of the BARR-C Coding Standards

- **Switch Statements Rule:**
  - The **break** for each case shall be **indented to align with** the **associated case**, rather than with the contents of the case code block.
  - All **switch statements** shall contain a **default block**.
  - **Any case designed to fall through to the next shall be commented to clearly explain the absence of** the corresponding **break**.

```
switch (err){
        case ERR_A:
        ...
        break;
        case ERR_B:
        ...
        // Also perform the steps for ERR_C.
        case ERR_C:
        ...
        break;
        default:
        ...
        break;
}
```

Some of the BARR-C Coding Standards

- **Loops Rule:**
  - **Magic numbers shall not be used** as the **initial value or in the endpoint test** of a while, do…while, or for loop.
  - With the exception of the initialization of a loop counter in the first clause of a for statement and the change to the same variable in the third, **no assignment shall be made in any loop's controlling expression**.
  - **Infinite loops** shall be implemented via controlling expression **for (;;)**.
  - Each **loop with an empty body** shall feature a set of braces enclosing a **comment to explain why nothing needs to be done** until after the loop terminates.

```
// Why would anyone bury a magic number (e.g., "100") in their code?
for (int row = 0; row < 100; row++)
{
        // Descriptively-named constants prevent defects and aid readability.
        for (int col = 0; col < NUM_COLS; col++)
        {
                ...
        }
}
```

Some of the BARR-C Coding Standards

- **Jumps Rule:**
  - The **use of goto** statements shall be **restricted**.
  - C Standard Library functions **abort()**, **exit**(), **setjmp**(), and **longjmp**() **shall not be used**.

Some of the BARR-C Coding Standards

- **Equivalence Tests Rule:**
  - When **evaluating the equality of a variable against a constant**, the **constant shall always be placed to the left** of the equal-to operator (==).

```
int  p;
If (p == 0)
{
        return (ERROR);
}
```

```
int  p;
If (0 == p)
{
        return (ERROR);
}
```

Code review

- **Code review for a module is carried out after** the module is **successfully compiled** and the all the syntax errors have been eliminated.
- **Code reviews** are extremely **cost-effective strategies for reduction in coding errors** and to produce high quality code.
- **Types of reviews**
  1. **Code inspection: examination of code for the presence of certain kinds of errors**
  2. **Code walk through: hand simulation of code execution**

Code Inspection

- Aim is to discover some common types of errors caused **due to oversight and improper programming**.
- In other words**, during code inspection** the code is examined for the presence of certain kinds of errors
  - **For instance**, consider the classical error of **writing a procedure that modifies a formal parameter while the calling routine calls that procedure with a constant actual parameter**.
  - **void fn(const int); fn(x); void fn(const int x){x=y;}**
- In addition to the commonly made errors, **adherence to coding standards** is also checked during code inspection.
- **Good software development companies** collect statistics regarding **different types of** errors commonly committed **by their engineers** and identify the type of errors most frequently committed.
- Such a list of **commonly committed errors can be used** during code inspection **to look out for possible errors**.

Code Inspection

- list of some **classical programming errors** which can be **checked during code inspection**
  - Use of **uninitialized** variables.
  - **Jumps into loops.**
  - **Non-terminating loops.**
  - **Incompatible assignments.**
  - **Array indices out of bounds**.
  - **Improper storage allocation and deallocation**
  - **Mismatches** between **actual and formal parameter** in procedure calls.
  - **Use of incorrect logical operators or incorrect precedence among operators**.
  - **Improper modification of loop variables**.
  - **Comparison** of **equality of floating point variables**, etc.

Code Walk Through

- It is an **informal code analysis** technique.
- A few members of the development team are given the code few days before the walk through meeting to read and understand code.
- Each **member selects some test** cases and simulates execution of the code by hand (i.e. trace execution through each statement and function execution).
- The **main objectives** of the walk through
  - **To discover the algorithmic and logical errors** in the code
- code walk through guidelines
  - **team** performing code walk through **should not be either too big or too small**
  - Discussion should **focus on discovery of errors** and **not on how to fix the discovered errors**
  - **avoid** the **feeling among engineers that they are being evaluated** in the code walk through meeting
  - **managers should not attend the walk through meetings**

Clean room testing

- It is the **strategy used by IBM** to **produce zero-defect software**.
- This type of testing relies heavily on walk through, inspection, and formal verification.
- The **programmers are not allowed to test** any of their code **by executing the code** other than doing some syntax testing using a compiler.
- The underlying philosophy is based on avoiding software defects by using a rigorous inspection process.
- The clean room approach to software development is based on **five** characteristics
- The main problem with this approach is that testing effort is increased as walk through, inspection, and verification are time-consuming.

Clean room testing

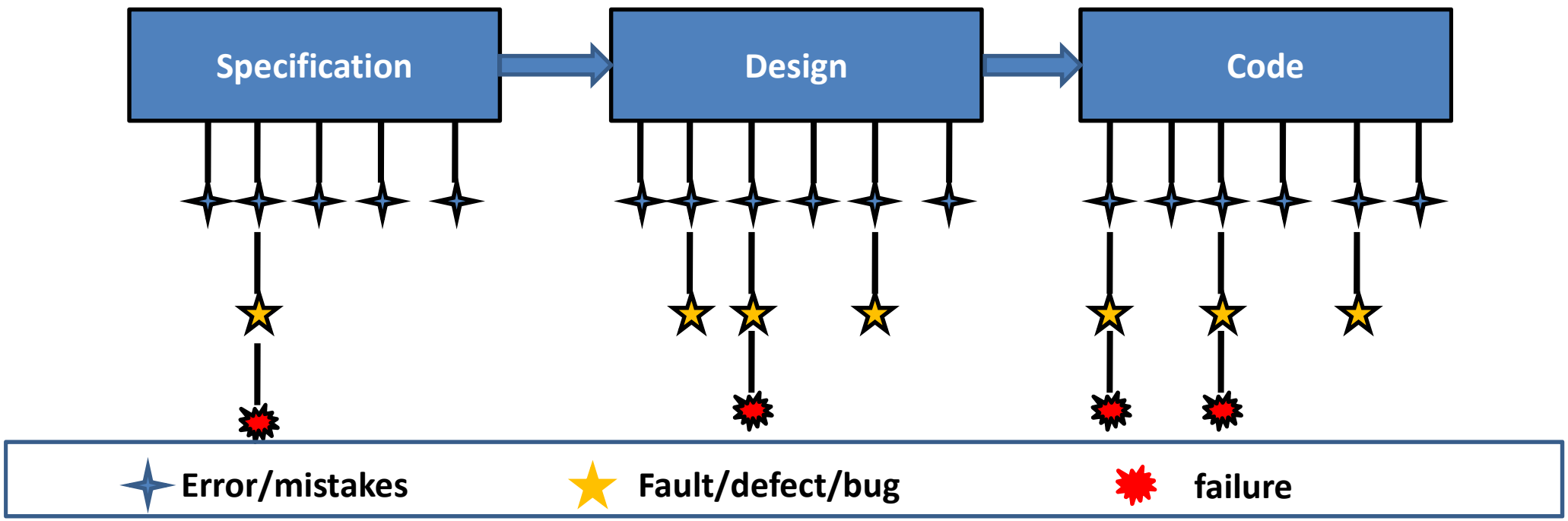
- **Formal specification**:
  - The software to be developed is formally specified. A state-transition model which shows system responses to stimuli is used to express the specification.
- **Incremental development**:
  - The software is partitioned into increments which are developed and validated separately using the clean room process. These increments are specified, with customer input, at an early stage in the process.
- **Structured programming**:
  - Only a limited number of control and data abstraction constructs are used. The program development process is process of stepwise refinement of the specification.
- **Static verification**:
  - The developed **software is statically verified using rigorous software inspections**. There is no unit or module testing process for code components.
- **Statistical testing of the system**:
  - The integrated software increment is tested statistically to determine its reliability. These statistical tests are based on the operational profile which is developed in parallel with the system specification.

Faults and failures

- **Programming need intensive human effort**
- Whether expert or beginner, all programmers commit mistakes
  - Even **experts programmers are estimated to add 50 bugs/1000 lines of code** (LoC)
  - **Extensive testing eliminates most of them but still 1 bug/1000 LoC may remain**
- A **program may fail because of presence of defect** or bug
- **Mere presence of a bug/fault may not always lead to failure**

Some definitions

- **Error**, **Fault**, **Failure**
- As per IEEE standard 1044 (Yr 1993), **error, bug and fault are synonymous**.
- As per recent standard, definitions of these terms are refined



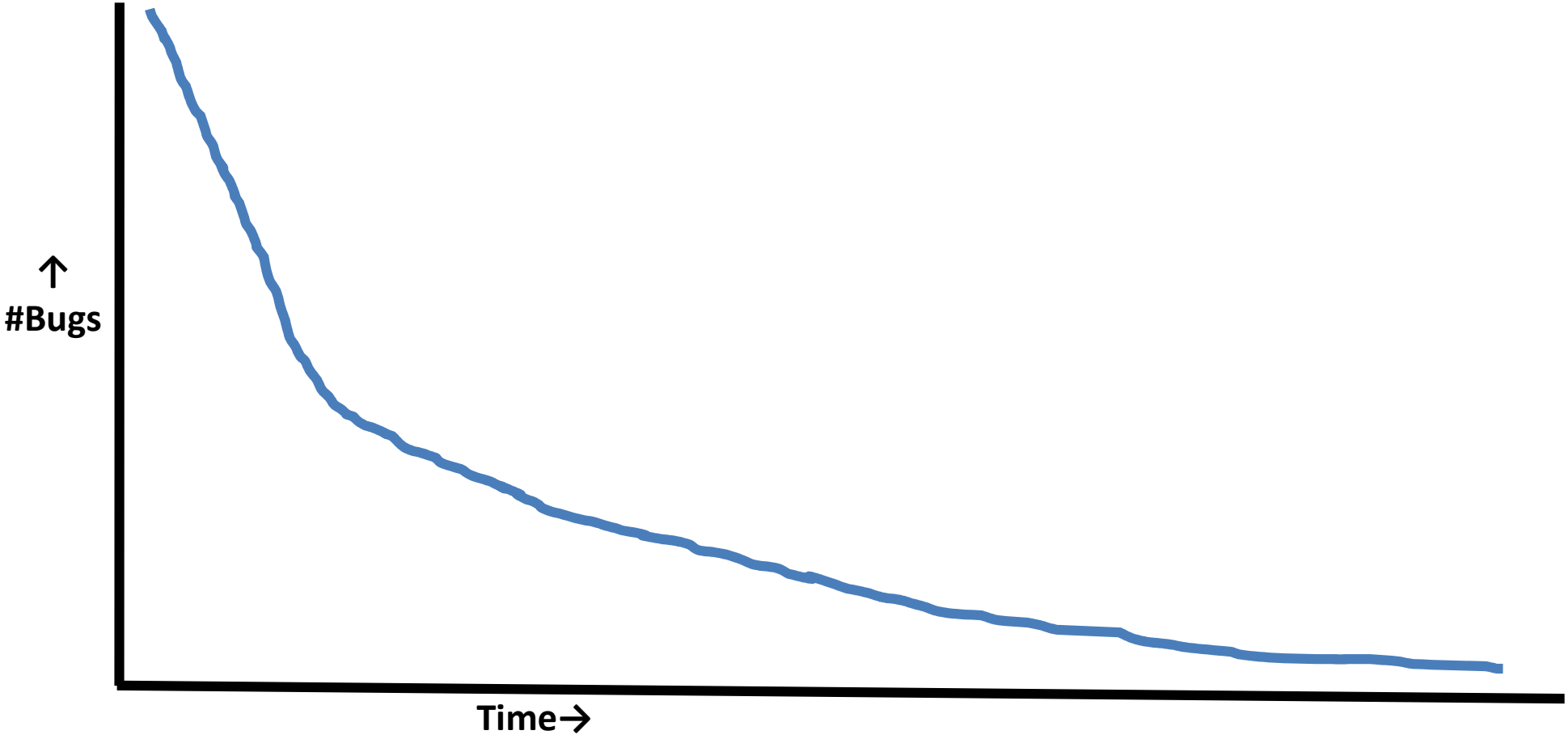
Infamous Bugs

- Around **60% of bugs** in S/W are **because of poor Designing** and remaining **40%** are **due to poor implementation**
- One of the **most infamous bugs** is:
  - The **Explosion of the Ariane 5** (June 4, 1996): Unmanned **Ariane 5 rocket** launched by the European Space Agency exploded just forty seconds after its lift-off from Kourou, French Guiana. **Total loss were valued over 370 million of US dollars**.
    - The cause of the failure was a software error in the inertial reference system. Specifically a **64 bit floating point number relating to the horizontal velocity of the rocket** with respect to the platform **was converted to a 16 bit signed integer**. The number was larger than 32,767, the largest integer storable in a 16 bit signed integer, and thus the conversion failed.

Some Facts on Testing

- **How to test?**
  - **Give In Put** to the program
  - **Get** the **Out Put**
  - Check if the **Out Put** is **as expected**?
  - If **Out Put** is **not as expected**, **generate a test report to find out** why **program failed**
  - **Locate** the **source** of the **error** and **fix it**
- **Testing accounts for lion share of S/W development cost**
- **Testing takes less time** in compare to **other phases because of parallelism** in **testing process**

How long to test?



- When **no more bugs** are reported
- **Seeding** of **bugs**

Some Facts on Testing

- **Earlier testing process** was to give some set of input to check if the system performs as expected
  - A kind of **monkey testing**
  - **No need of expertise** in **sophisticated techniques**
- Testing process is becoming more systematic
  - **New testing techniques/tools are being developed**
  - **Automation of testing process**
- **Testing is essential in all the phases of the development life cycle**

Verification and Validation

- **Verification**
  - is the process of determining **whether the output of one phase of software development conforms to that of its previous phase:**
  - **Approaches used: Review, Simulation, unit testing, integration testing etc.**
  - **Done by developers**
- **Validation**
  - is the process of determining whether a **fully developed system conforms to its requirements specification (SRS)**
  - **Approaches used: System testing.**
  - **By Test Engineers**
- **Verification** is **concerned with phase containment of errors**, Whereas **validation is to** make the **final product be error free**.

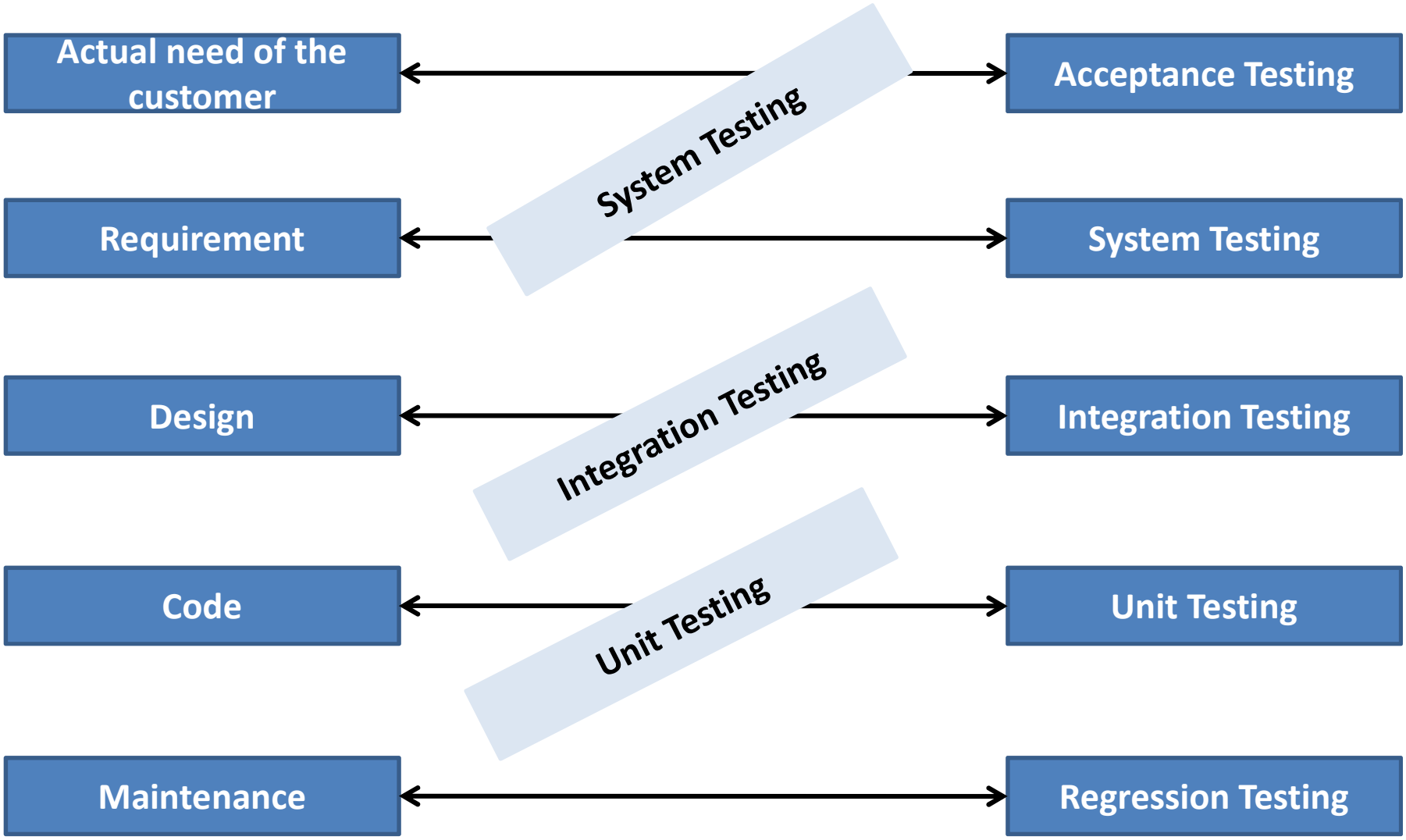
Test Levels

- **Software are tested at four levels**
  - **Unit Testing**
    - Once a module is completed
  - **Integration testing**
    - After integrating a module in to the software
  - **System testing**
    - If the system as a whole it works
  - **Regression Testing**
    - During maintenance

Test Levels

- **Unit Testing**
  - **Test each module** or component independently
  - Mostly **done by developers** of the module
- **Integration and System testing**
  - **Test the system as individual modules are developed and integrated** in to the system till completion of the development
  - It is to **ensure proper function of interfaces, parameter states, runtime exception handlings**
  - Done by **separate testing or QA (quality assurance) team**
  - **Smoke testing**: **Partial System testing** done frequently in the industry
- **Acceptance Testing**
  - **A kind of system testing**
  - Validation of the system functions **by the customer**

Levels of Testing



Testing Activities

- Test suit design
- Run Test cases
- Check results to detect failures
- Prepare the failure list

Testing Team

- Debug program to locate the errors
- Correct the errors

Development Team

Types of System Testing

- Based on **Types of Testing**
  - **Functionality Testing**
    - Check if the **functionalities specified in the SRS** are correct
  - **Performance testing**
    - Check if the **non-functionalities specified in the SRS** are correct

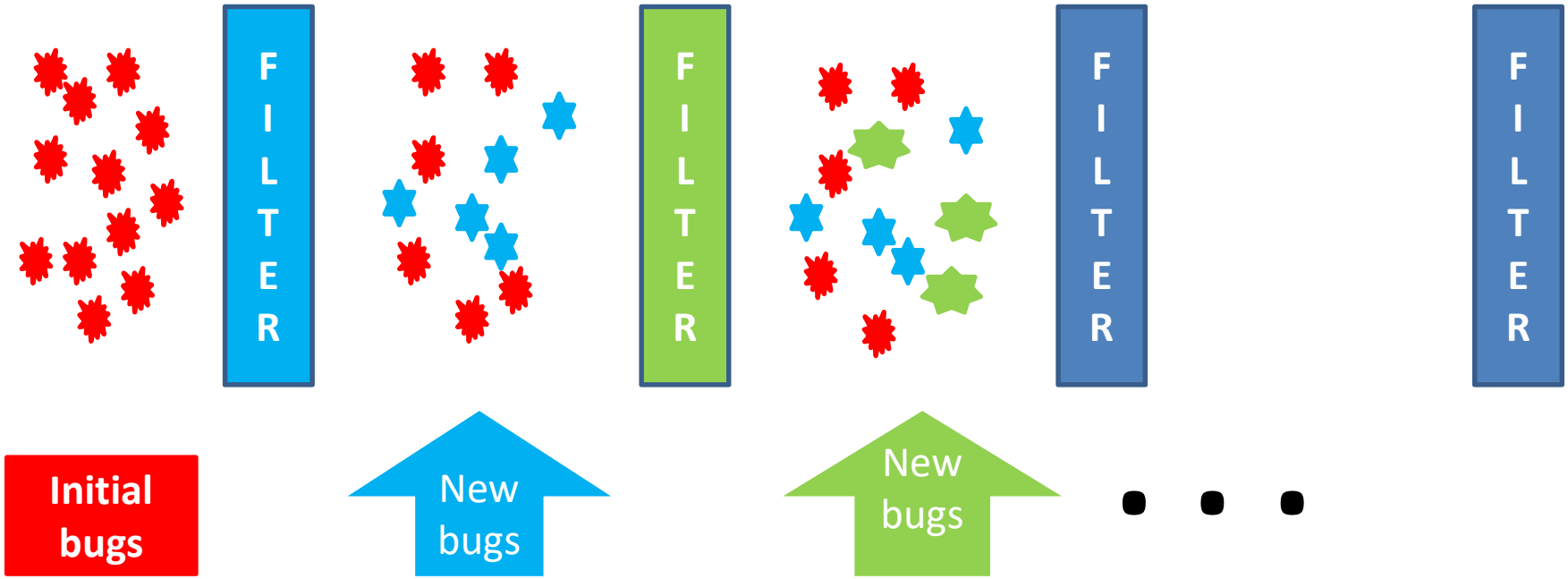
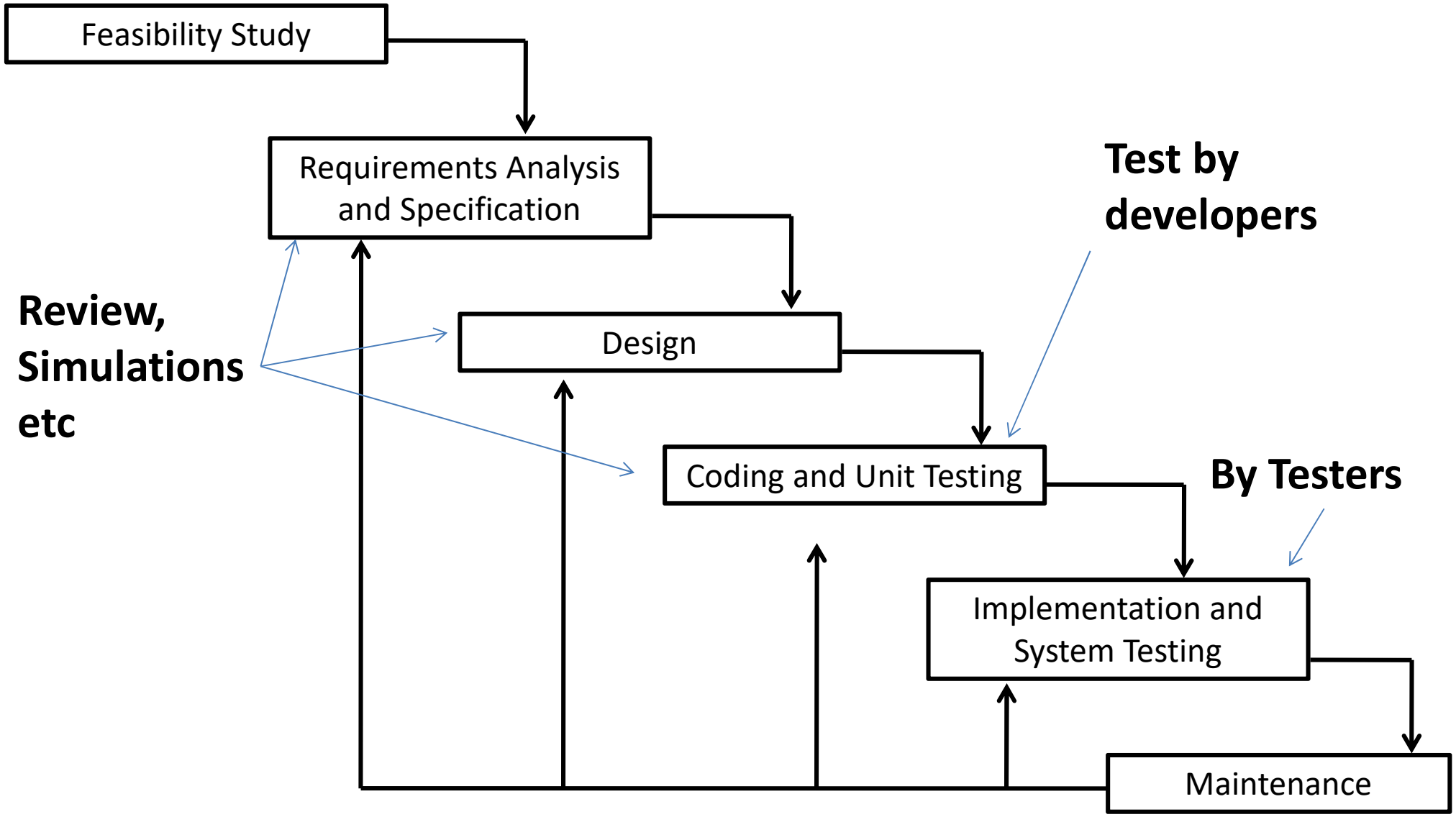
- Based on **Who performs the testing**
  - **Alpha** testing
    - By the development team
  - **Beta** testing
    - Friendly set of customers
  - **Acceptance** testing
    - By the actual customer

Types of System Testing

- **Performance testing**
  - **Response time**
  - **Throughput**
  - **Usability**
  - **Stress**
  - **Recovery**
  - **Configurations**
  - etc.

Pesticide effect

- Errors escaped one detection technique can not be detected any more by repeating the same technique.
- Rectifying one error may add new errors.
- Research by **Capers Jones suggests only upto 30% bugs present can be eliminated by one step**



Use of test team in Waterfall model